

Morphic Studio

Document 27 — Authentication & Identity System (Expanded Specification)

Document ID: MS-027

Version: 2.0

Status: Expanded Specification

Priority: Core Platform Infrastructure

Purpose

The Authentication & Identity System manages user identities, authentication, authorization, sessions, workspace access, and account security throughout Morphic Studio.

It ensures that every action performed within the platform can be securely associated with an authenticated user while providing a seamless login experience.

Although Version 1 targets a small private group of users, the architecture is designed to support future public deployment without requiring major redesign.

Vision

Authentication should be:

- Secure
- Fast
- Reliable
- Invisible during normal use

Creators should spend their time creating—not repeatedly signing in or managing credentials.

Core Philosophy

Identity is the foundation of trust.

Every project, asset, workflow, approval, and creative decision should always have a verifiable owner.

Objectives

The Authentication System shall:

- Authenticate users.
 - Authorize access.
 - Manage identities.
 - Protect accounts.
 - Support multiple login methods.
 - Manage sessions.
 - Secure API requests.
 - Enable future enterprise features.
-

Identity Model

Each user has one global identity.

That identity may belong to:

- Personal Workspace
- Multiple Team Workspaces
- Future Organization Workspaces

One identity.

Many projects.

Many workspaces.

Authentication Methods

Version 1

- Email + Password

Future Versions

- Google
- GitHub
- Microsoft
- Apple
- Passkeys (WebAuthn)
- Enterprise SSO
- Magic Links

The system should be extensible so additional providers can be added without changing existing accounts.

Registration Flow

User enters:

- Name
- Email
- Password

↓

Email verification

↓

Account created

↓

Personal Workspace created automatically

↓

Welcome flow

↓

Dashboard

Login Flow

Email



Password



Credential validation



Session created



Security checks



Dashboard

Password Policy

Passwords should:

- Meet minimum length requirements.
 - Encourage strong entropy.
 - Never be stored in plain text.
 - Be hashed using modern password hashing algorithms.
 - Support future passwordless authentication.
-

Session Management

Each active session stores:

- Session ID
- User ID
- Device

- Browser
- Login Time
- Last Activity
- Expiration Time
- IP Information (where appropriate)

Users may revoke active sessions.

Authorization

Authentication answers:

"Who are you?"

Authorization answers:

"What are you allowed to do?"

Permissions are evaluated using:

Workspace

↓

Role

↓

Permission Set

↓

Resource

↓

Requested Action

Account Recovery

Supported recovery includes:

- Password reset email
- Email verification
- Session invalidation
- Future MFA recovery options

Recovery should balance usability with security.

Account Settings

Users may manage:

- Profile
 - Display Name
 - Avatar
 - Email
 - Password
 - Connected Providers (future)
 - Sessions
 - Notifications
 - Language
 - Time Zone
-

User Stories

As a creator, I want to securely access my projects from any supported device.

As a workspace owner, I want to control who has access to my projects.

As a user, I want to recover my account if I forget my password.

As an administrator, I want secure authentication logs.

Functional Requirements

The Authentication System shall allow users to:

- Register.
 - Log in.
 - Log out.
 - Reset passwords.
 - Verify email.
 - View active sessions.
 - Revoke sessions.
 - Update credentials.
 - Manage account settings.
-

Security Measures

The system should include:

- Secure password hashing.
- Session expiration.
- Rate limiting.
- Login attempt monitoring.
- CSRF protection where applicable.
- Secure cookies.
- HTTPS-only communication.
- Input validation.

Future enhancements may include:

- Multi-factor authentication.
 - Device trust.
 - Risk-based authentication.
 - Passkeys.
-

Authentication Events

Important events include:

- Registration
- Email verification
- Login
- Logout

- Password change
- Password reset
- Failed login
- Session revoked
- New device login

These events should be logged for auditing.

UI Components

The interface includes:

- Login Page
 - Registration Page
 - Forgot Password Page
 - Reset Password Page
 - Verify Email Page
 - Account Settings
 - Active Sessions
 - Security Settings
-

Database Entities

Core entities include:

- User
 - Session
 - AuthenticationProvider
 - VerificationToken
 - PasswordResetToken
 - SecurityEvent
 - LoginHistory
-

API Considerations

Potential endpoints include:

- Register User
- Verify Email
- Login
- Logout

- Refresh Session
 - Forgot Password
 - Reset Password
 - Retrieve User
 - Update User
 - Revoke Session
-

Non-Functional Requirements

The Authentication System should:

- Scale with future growth.
 - Maintain low login latency.
 - Protect against common web attacks.
 - Support future authentication providers.
 - Remain highly reliable.
-

Error Handling

If authentication fails:

- Display a generic error.
- Avoid revealing whether an email exists.
- Log the attempt.

If verification tokens expire:

- Allow regeneration.
- Notify the user.

If suspicious activity is detected:

- Record the event.
 - Notify the user when appropriate.
 - Support future automated responses.
-

Future Expansion

Future versions may include:

- Multi-factor authentication.

- Passwordless login.
 - Biometric authentication.
 - Hardware security keys.
 - Enterprise identity federation.
 - Single Sign-On (SSO).
 - Fine-grained access policies.
-

Acceptance Criteria

The Authentication & Identity System succeeds when creators can securely access their work with minimal friction while the platform reliably protects identities, sessions, and permissions.

Key Principle

Authentication should strengthen trust without interrupting creativity. Every creator should feel that their identity and work are protected while enjoying a smooth and intuitive sign-in experience.

End of Document 27